Project Report

50.035 Computer Vision

Team 06 / Toh Jia Le (1007004)

27 April 2026

Payload Loss Detection Using Computer Vision

Design/Capstone Track

Background: Capstone project involves an autonomous logistics robot transporting tools, toolboxes, equipment and supplies within an industrial building. During transport, payloads may shift/fall off due to vibrations, or get removed by unauthorized personnel. To maintain security, accountability and operational efficiency, there is a need for a real-time payload monitoring system. Initial concepts involved an RFID system, but due to high associated costs and technical complexity, there was a pivot towards a computer vision-based approach. Computer vision also required less modification to the payload shelf while offering greater flexibility for future software improvements.

Objective: Implement a real-time, computer vision-based payload loss detection system for a logistics robot to monitor payloads during transport.

Requirements:

1. Real-Time Processing: The system must be able to detect any payload loss within 3 seconds of occurrence.
2. Payload State Classification: The system must accurately classify ($\geq$ 95%) payload status into two categories: “Payload Present” and “Payload Absent”.

Constraints:

1. Hardware Limitation: The system must run on the robot’s current onboard computer (Advantech MIO-5393 with integrated graphics) without throttling its primary navigation functions.
2. Data Privacy: Due to the sensitive/restricted nature of the industrial environment, all video frames must be processed in memory and not be stored permanently.

System Design: Hardware

Hardware design focused on achieving sufficient visual coverage for payload monitoring without disrupting the shelf's main loading/unloading functions. Although higher camera positions provided greater coverage, they interfered more with loading/unloading. Therefore, the lowest camera height was selected despite its reduced coverage (See Figure 1).

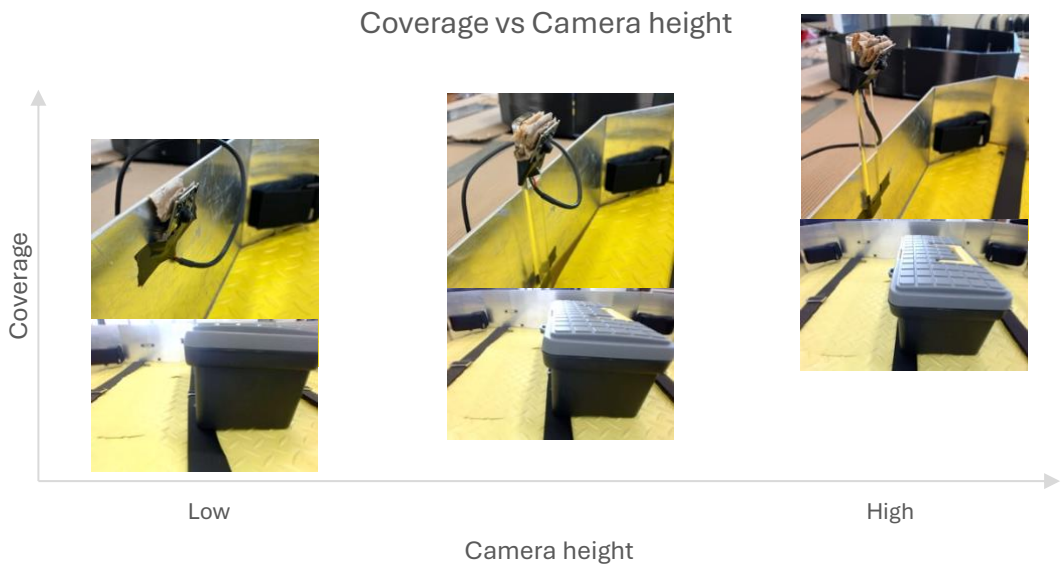

Figure 1: Coverage vs camera height graph

To compensate, four cameras were placed on the front, back, left and right sides of the payload shelf to reduce occlusion and improve overall visibility of the payload area (See Figure 2), and payload changes can still be observed even when one view is blocked. Cameras with different fields of view (FOV) were also evaluated, and the final choice was the 90º FOV camera that provided the widest coverage among the tested options while maintaining the smallest camera body for practical integration (See Figure 3).

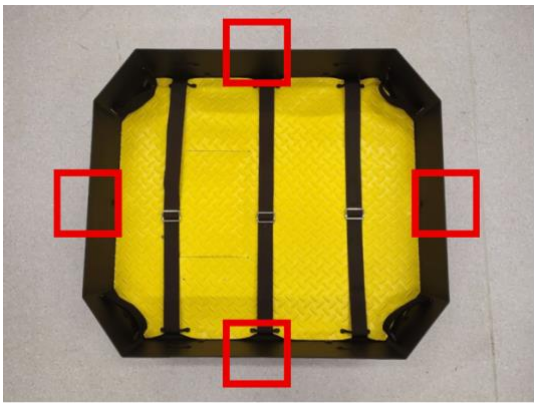

Figure 2: Payload shelf with final camera positions

Figure 3: Final 90º FOV camera with custom mount

System Design: Software

General Approach: As shown in Figure 4, the system takes in four live video feeds from the front, back, left and right cameras. The feeds are then run through the CV algorithm or model, followed by post-processing, and outputs either "Payload Present" or "Payload Absent".

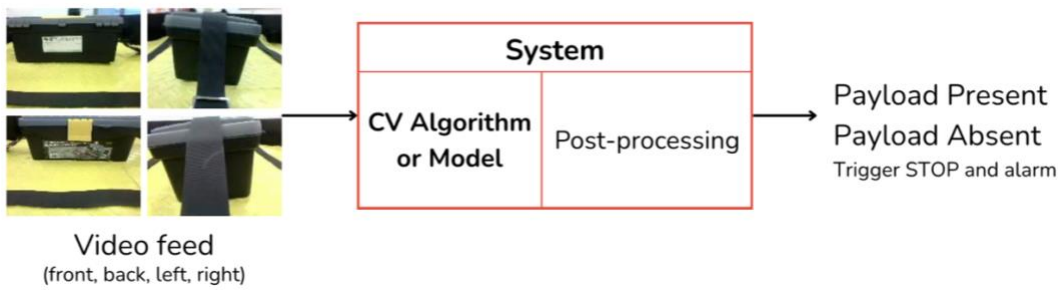

Figure 4: Software pipeline for payload loss detection

The project evaluated three different approaches for the "System" component, from classical computer vision algorithms with lower computational cost, to heavier deep learning-based methods. A test dataset was then used to quantify the performance of each approach before selecting the most suitable method for the system's hardware constraints.

Factor	**Count**	**Description**
Lighting conditions	3	Indoor low, medium and full brightness
Payload scenarios	6	Payload scenarios with different items: Toolbox, tools, bottles, etc.
Event classes	2	Payload loss event vs Normal operating conditions with no payload loss
Camera views	4	Front, back, left, right
Total videos	144	

Figure 5: Composition of test dataset

Test Dataset: The test dataset consists of 144 videos in total (See Figure 5), with each video approximately 30 to 40 seconds long. A ground truth CSV file was also created for evaluation (see Figure 6), to compare predictions against the actual payload state and event timing.

Column	**Description**
filename	Name of the video file
camera_id	Camera view: front, back, left or right
is_loss_event	Indicates whether a payload loss event occurs in the video (0 = no, 1 = yes)
loss_frame	Frame number at which payload loss occurs (-1 if no loss event occurs)
total_frames	Total number of frames in the video

Figure 6: Columns in ground truth CSV file

Approach #1: Structural Similarity Index (SSIM)

The first approach used Structural Similarity Index (SSIM) to compare the initial and current frames, and detect visual changes in the payload region. This method was considered because it is lightweight and has low computational cost, making it suitable for the robot's onboard hardware.

However, initial testing found it to be highly sensitive to lighting changes and shadows, which caused fluctuations even when no payload loss had occurred. It was also difficult to determine a reliable threshold for classification, especially for small objects where the visual difference was less pronounced. As a result, this approach was deemed to not be sufficiently robust for this application.

Approach #2: YOLO26n

The second approach used YOLO26n, a state-of-the-art object detection model with ~2.4m parameters that can provide object counts. This method was considered because it was more advanced than SSIM, and the initial hypothesis was that exact object classification accuracy was not critical, as long as the total object count in each frame was correct, so that the initial and current object counts could be compared to detect payload loss.

The model was used with its original pre-trained weights without fine-tuning, as fine-tuning was not practical due to the large effort required to collect and annotate training data across many possible payload and tool types, as well as the lack of access to the exact items that would be carried during deployment. As a result, detection was limited to the object classes available in the COCO dataset. This created a major limitation, as important payload items such as toolboxes could not be detected reliably. In addition, object counts fluctuated under different lighting conditions, which reduced reliability for payload monitoring.

Approach #3: Siamese Network

The third approach used a custom Siamese network, which was inspired by the idea that payload loss detection is fundamentally a visual comparison task. Hence this model was designed to compare the current payload state against the initial state of the shelf, before outputting a payload state classification score (where 1 represents payload loss).

Frames are passed through a frozen MobileNetV3-Small feature extractor to obtain feature vectors f_1 and f_2, where f_1 is the feature vector of the initial frame and f_2 is the feature vector of the current frame. f_1 is cached so that it does not need to be recomputed for every frame, reducing computational cost. f_2 is recalculated every frame for the latest payload state. The absolute difference $|f_1 - f_2|$ and the element-wise product of $f_1 \odot f_2$ are then concatenated

before being passed into a multilayer perceptron (MLP) classifier head. The final output is a score from 0 to 1 indicating the likelihood of payload loss (See Figure 7).

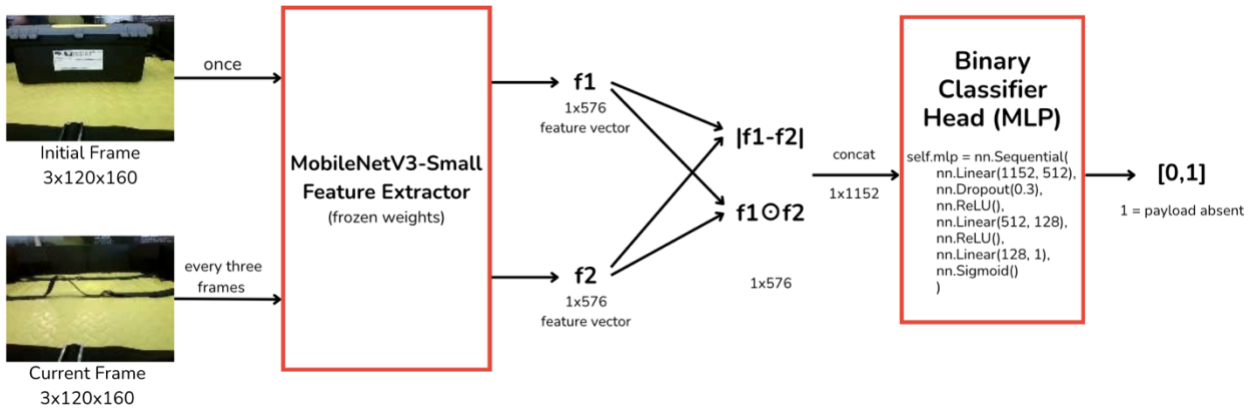

Figure 7: Architecture of the Siamese network used

The absolute difference gives the model information about how much the current frame differed from the initial reference frame, while the element-wise product captures feature similarity between the two embeddings—combining both gives the classifier information about both similarity and difference.

For this approach, a separate training dataset was created using 104 payload loss videos and 32 normal operation videos. The dataset is intentionally imbalanced because normal-operation frame pairs can also be obtained from payload loss videos (as long as both frames are taken from before/after the loss event), reducing the number of normal operation videos needed.

Two training variations were evaluated: one without data augmentation, and one with augmentation applied to the current-frame input to improve robustness against lighting and minor camera variations.

The extracted dataset was then split into training and validation sets using an **80/20 split** while maintaining class distribution. A weighted binary cross-entropy loss was used during training to account for class imbalance and ensure that payload loss events were not underrepresented in the learning process. Early stopping with a patience of 15 epochs was applied to prevent overfitting and to select the best-performing model based on validation loss.

Training results showed that **the model without data augmentation performed better**, achieving a lower validation loss of 0.068764 compared to 0.113096 for the augmented model. This suggests that the existing dataset already captured sufficient variation in lighting and scene conditions. As a result, the non-augmented model was selected for final evaluation on the test dataset.

Evaluation

The three approaches were evaluated on the test dataset using a combination of event and frame-level metrics as well as performance metrics. These metrics were selected to reflect both detection reliability and practical deployment constraints. Event-level metrics assess, at the video level, whether payload loss events are detected reliably, while frame-level metrics evaluate the accuracy of predictions over time within each video. Detection delay measures how quickly a method responds after a loss event occurs, and inference latency reflects the computational efficiency of each approach for real-time deployment.

1. Event-Level Recall

Event-Level Recall measures the system's ability to detect payload loss events at the video level. For each video containing a payload loss event, a detection is considered correct if the system flags a loss at least once at or after the ground truth loss frame. This metric reflects the system's ability to avoid missed detections.

Event-Level Recall = No. of correctly detected loss events / Total no. of actual loss events

2. Event-Level Precision

Event-Level Precision measures how many detected events correspond to actual payload loss events. A prediction is considered a detected event if the system flags a loss in a video. This metric captures the tendency of the system to raise false alarms at the video level.

Event-Level Precision = No. of correctly detected loss events / Total no. of detected events

3. Frame-Level Precision

Frame-Level Precision evaluates the accuracy of loss predictions at the frame level across all videos. It measures how often frames predicted as "payload loss" are actually correct. This is important for minimizing unnecessary interruptions during operation.

Frame-Level Precision = True Positive Frames / False Positive Frames

4. Detection Delay

Detection Delay measures how quickly the system identifies a payload loss after it occurs. It is defined as the difference between the first detected loss frame and the ground truth loss frame, for videos where a correct detection occurs.

Detection Delay = Detected frame no. – Ground truth Loss frame

Lower detection delay indicates faster response, which is better for faster alerts.

5. Inference Latency

Inference Latency measures the time taken to process each frame, reported in milliseconds. This metric reflects the computational efficiency of each method and its suitability for real-time deployment on the robot's onboard hardware.

Inference Latency = Total Processing Time / Total No. of Frames

Lower latency indicates better real-time performance.

Overall Results

Method \ Metric	Event-Level Recall (%)	Event-Level Precision (%)	Frame-Level Precision (%)	Avg Detection Delay (frames)	Avg Inference Latency (ms/frame)
SSIM (threshold=0.80)	0.513889	0.293651	0.353561	-0.702703	0.561924
YOLO26n (conf=0.10)	0.208333	0.211268	0.518266	3.866667	25.537925
Siamese (no aug) (threshold=0.10, consec_frames=3)	0.763889	0.466102	0.504248	-1.018182	86.700773

Figure 8: Summary table comparing each approach against evaluation metrics

The Siamese network achieved the best overall performance among the three approaches, with the highest event-level recall and improved event-level precision compared to both SSIM and YOLO26n, indicating that it was more effective at reliably detecting payload loss events while minimizing false positives. In contrast, SSIM struggled with precision due to sensitivity to visual noise, while YOLO26n failed to generalize to the task due to mismatched object classes. Although the Siamese approach incurs the highest inference latency (~87 ms/frame), it is the only approach with acceptable detection accuracy. Hence overall, the Siamese network is the most suitable approach for this application.

Results: SSIM

SSIM performed poorly across the tested thresholds (See Figure 9). The best overall balance was achieved at a threshold of 0.80, with 51.4% event-level recall, 29.4% event-level precision, and 35.4% frame-level precision. Although SSIM was very fast at around 0.56 ms/frame, its low precision shows that it produced many false positives, making it unreliable for payload loss detection.

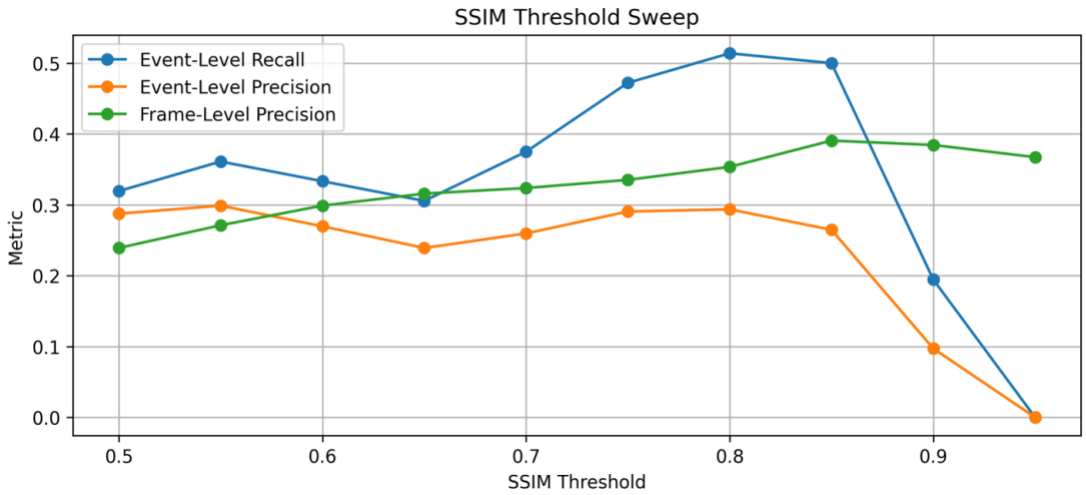

Figure 9: SSIM threshold sweep

Results: YOLO26n

YOLO26n also performed poorly, with the best recall achieved at a confidence threshold of 0.10 (See Figure 10). At this setting, it reached only 20.8% event-level recall, 21.1% event-level precision, and 51.8% frame-level precision. This suggests that YOLO26n missed most loss events, likely because the pretrained COCO classes did not match the actual payload items. Its latency was also much higher at around 25.5 ms/frame.

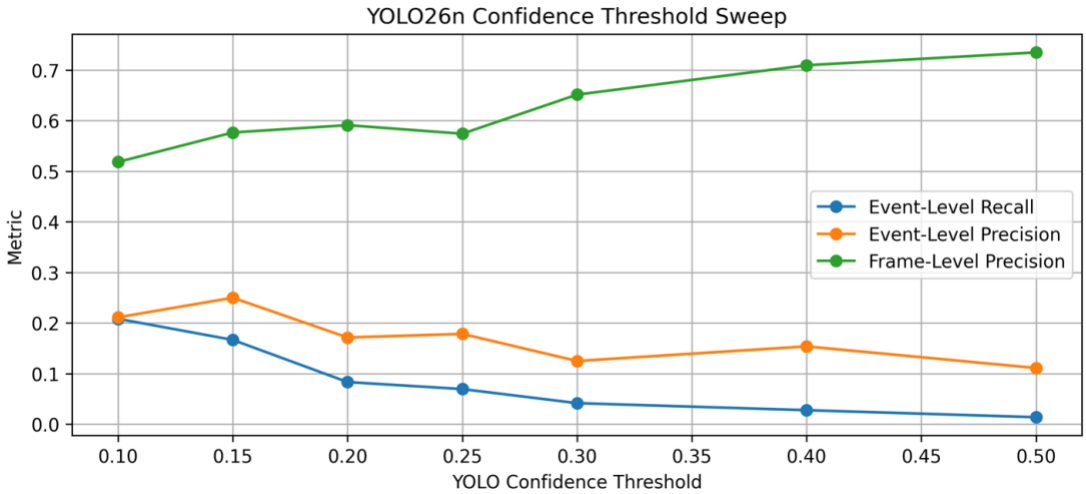

Figure 10: YOLO confidence threshold sweep

Results: Siamese Network

Figure 11 shows the training and validation loss curves for the Siamese network. The model converges steadily with a clear reduction in validation loss before early stopping, indicating stable learning without significant overfitting. The non-augmented model achieves a lower

validation loss compared to the augmented variant, and is therefore selected for final evaluation.

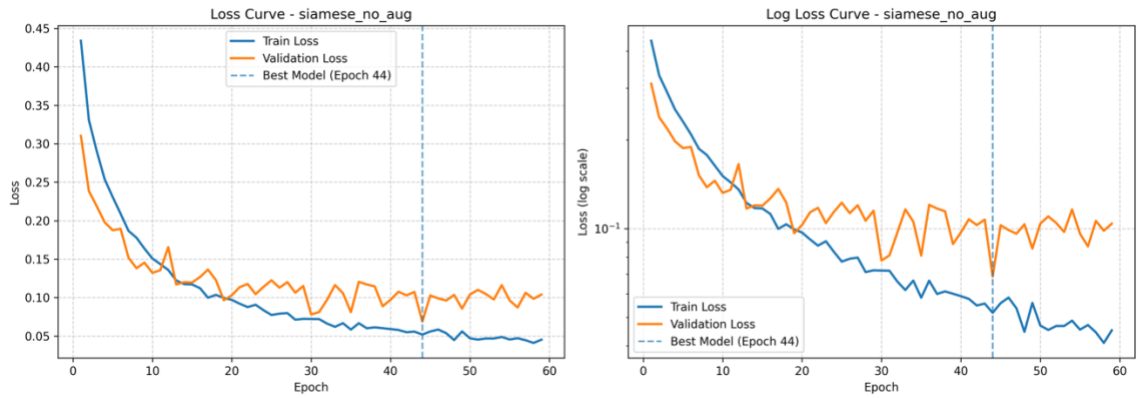

Figure 11: Siamese training and validation loss curves

Figure 12 presents the training and validation accuracy curves for both the augmented and unaugmented models. Both curves show consistent improvement during training, with validation accuracy closely tracking training accuracy. This suggests that the model generalizes well to unseen data and is not memorizing the training pairs.

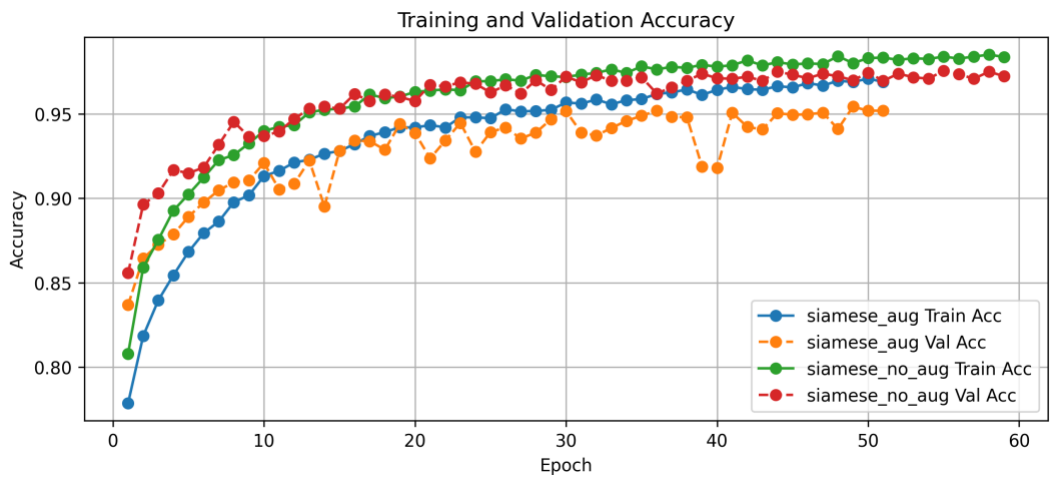

Figure 12: Siamese accuracy curves

The detection timeline for a sample video in Figure 13 shows that the Siamese network scores remain low throughout the video, with values staying well below the threshold of 0.5. This is expected, as there is no payload loss in this example. The model correctly maintains a low confidence for loss across all frames, indicating stable behavior and a low false-positive rate in normal operation.

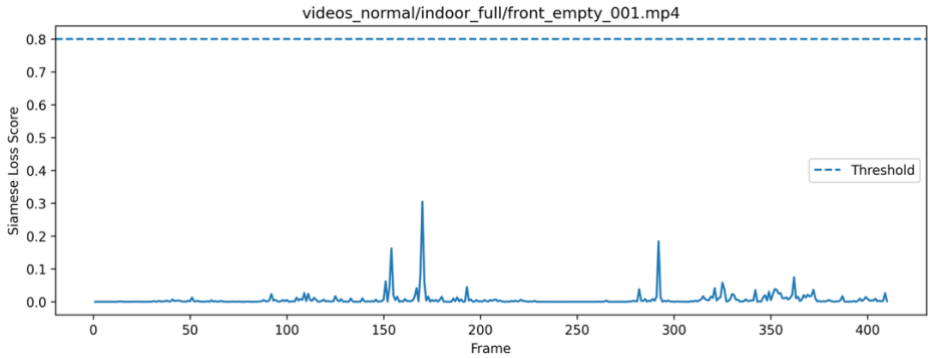

Figure 13: Siamese model detection timeline for a normal-operation video

Temporal filtering was applied by requiring multiple consecutive loss predictions, which improved robustness by reducing false positives from score fluctuations. Threshold and filtering parameters were selected based on the best trade-off between recall and precision (See Figures 14 and 15).

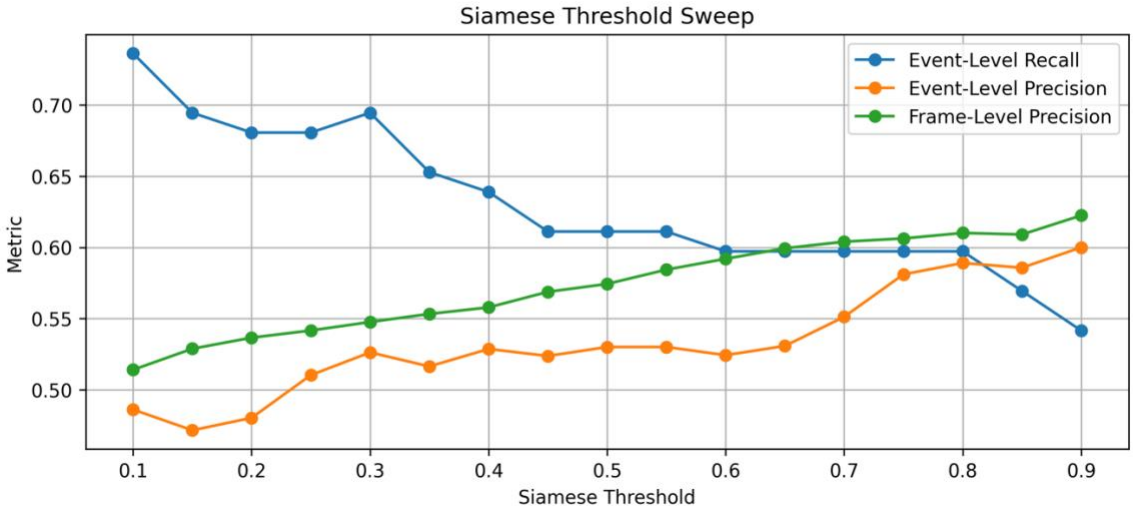

Figure 14: Siamese threshold sweep showing trade-off between metrics across thresholds

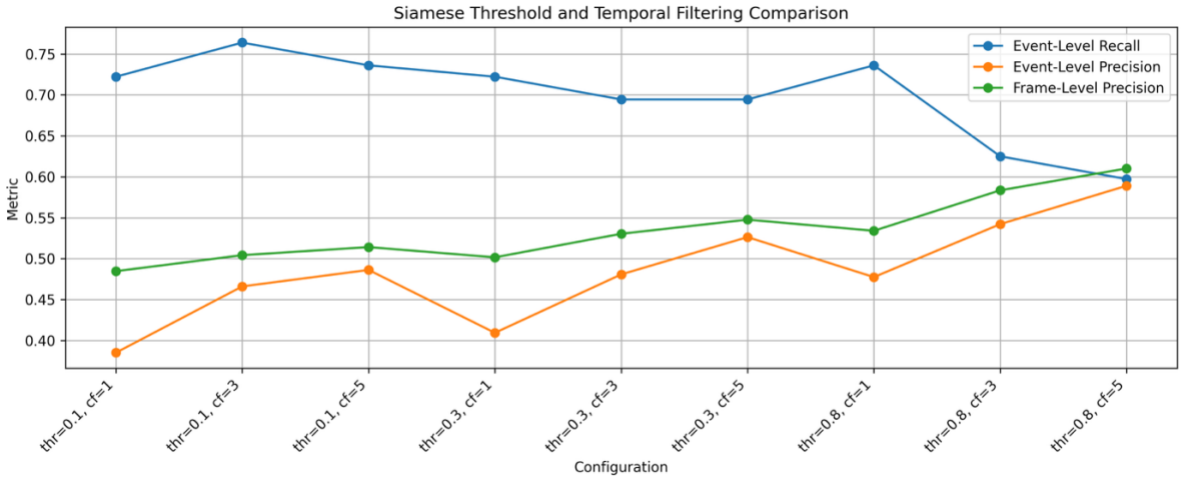

Figure 15: Siamese temporal filtering across selected thresholds and consecutive-frame requirements

The final configuration used a threshold of 0.10 with a consecutive-frame requirement of 3, achieving 76.4% event-level recall, 46.6% event-level precision, and 50.4% frame-level precision. Detection delay was low, with an average of -1.0 frames, indicating that the model often detected loss slightly before the annotated frame.

Discussion

The Siamese approach outperformed SSIM and YOLO26n because it directly models the visual difference between payload states, rather than relying on pixel similarity or object detection. SSIM was sensitive to environmental changes such as lighting and shadows, while YOLO26n struggled due to mismatched object classes.

Although Siamese approach incurs higher inference latency (~87 ms/frame), it provides significantly more reliable detection, making it more suitable for deployment in safety-critical environments.

There is still much room for improvement for the Siamese Network, however. Failure analysis shows that the Siamese network struggles more under low-light conditions. The highest number of failures occurs in videos_normal/indoor_low, indicating a tendency to produce false positives when visibility is poor (See Figure 16). This suggests that lighting variation introduces noise that affects the model's ability to distinguish between normal and loss states.

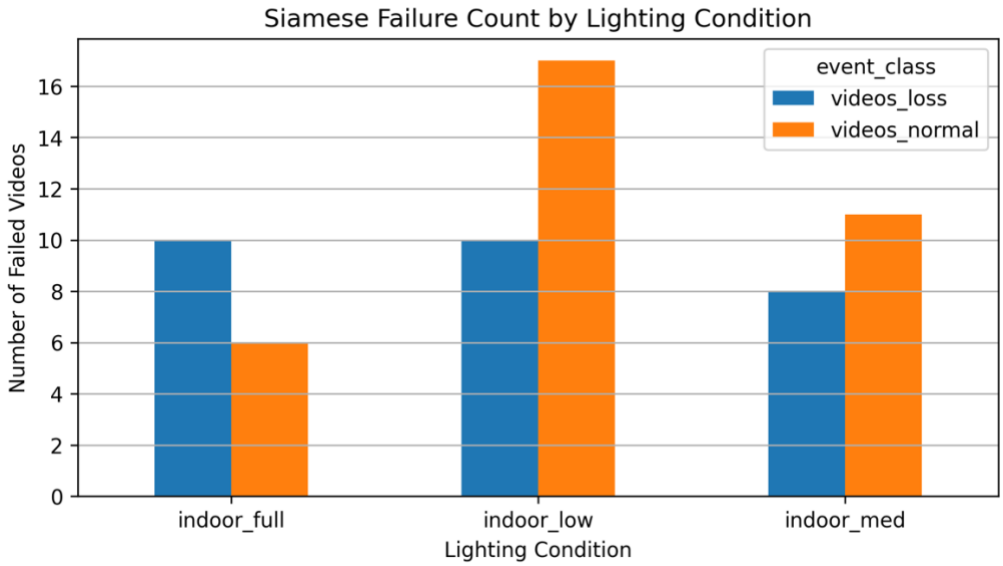

Figure 16: Number of failed videos by lighting condition for the Siamese network

For loss-event videos, failures are relatively consistent across lighting conditions (See Figure 16), indicating that missed detections are not strongly tied to lighting but may instead be due to subtle or gradual payload changes that are harder to distinguish from the reference state.

Overall, the results highlight that while the Siamese approach is effective, its robustness is influenced by environmental conditions, particularly low lighting, which remains a key area for improvement.

Impact and Potential Applications

The proposed payload loss detection system improves safety, security, and operational reliability for autonomous logistics robots. By monitoring payloads in real time, it can help detect dropped or missing items early, reducing the risk of misplaced equipment, operational delays, and safety hazards in industrial environments (specifically foreign object debris or FOD in aviation environments).

Beyond this project, similar payload monitoring systems could be applied to other transport platforms where carried items may shift, fall, or be removed during operation. Potential applications include last-mile autonomous delivery robots, drone logistics systems, and open-bed lorries or trucks.

Commercialization and Deployment Considerations

The system has potential for commercialization as an add-on safety and monitoring feature for autonomous transport platforms. Its value lies in improving accountability and reducing losses during transport, which are especially useful in industrial, warehouse and logistics settings.

However, deployment would require further engineering work to improve robustness under real operating conditions, and fine-tuning for specific use cases. This includes support for additional payload types, improved handling of lighting variation, and reliable multi-camera processing on embedded hardware. Additional integration with the robot's onboard software, alert system, and user interface would also be needed before real-world deployment.

Sustainability Impact

This project supports **SDG 8: Decent Work and Economic Growth** by promoting safer working environments and reducing risks associated with hazardous payload loss in industrial settings. It also supports **SDG 9: Industry, Innovation and Infrastructure** by enhancing logistics systems with AI-based monitoring and contributing to smarter industrial automation.

Future Work

Future work can focus on expanding the training dataset to cover more edge cases, such as extreme lighting changes and more varied payload combinations. Multi-camera stream processing should also be further optimized to improve runtime reliability.

Other next steps include improving robustness for multiple items, strengthening deployment on embedded hardware, and integrating the system more tightly with the robot's runtime response, such as automated alerts or emergency stop behaviour.

References

https://scikit-image.org/docs/stable/auto_examples/transform/plot_ssim.html

https://docs.ultralytics.com/compare/yolov10-vs-yolo26/

https://medium.com/@rinkinag24/a-comprehensive-guide-to-siamese-neural-networks-3358658c0513

https://aclanthology.org/P16-2022.pdf